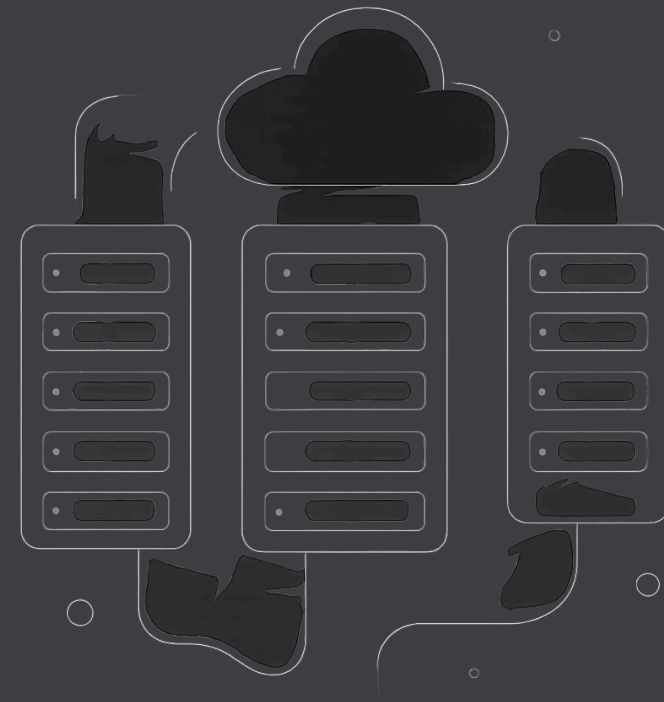


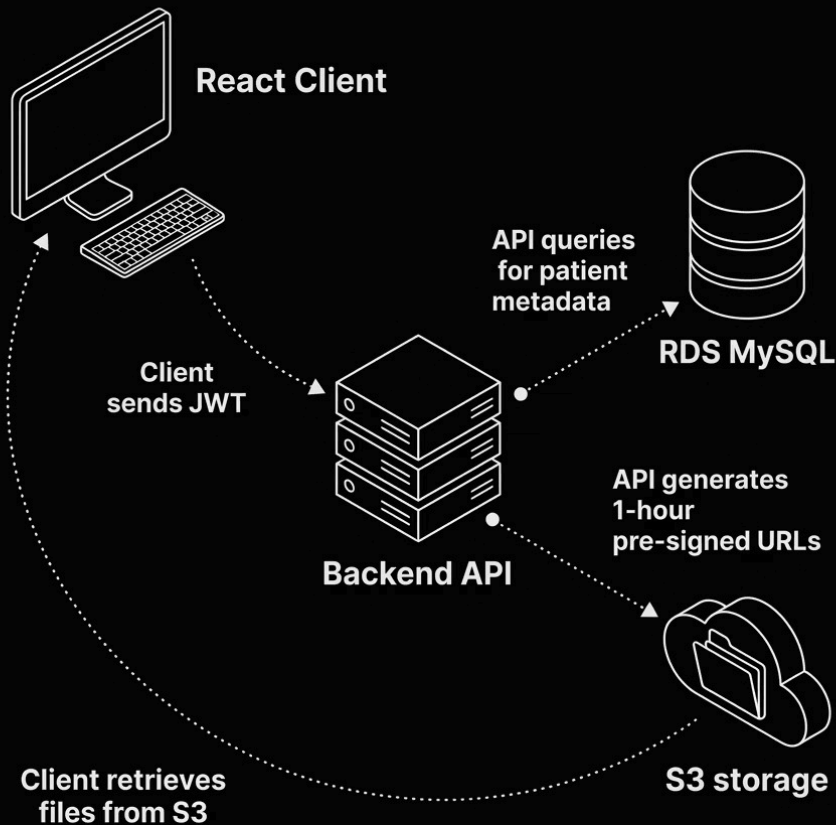
Infrastructure & Terraform Architecture Review

Highly-Available, Private VPC Deployment of the Patient
Management System

Technical review of network architecture, security controls, and IaC patterns —
covering Terraform-managed production infrastructure only.



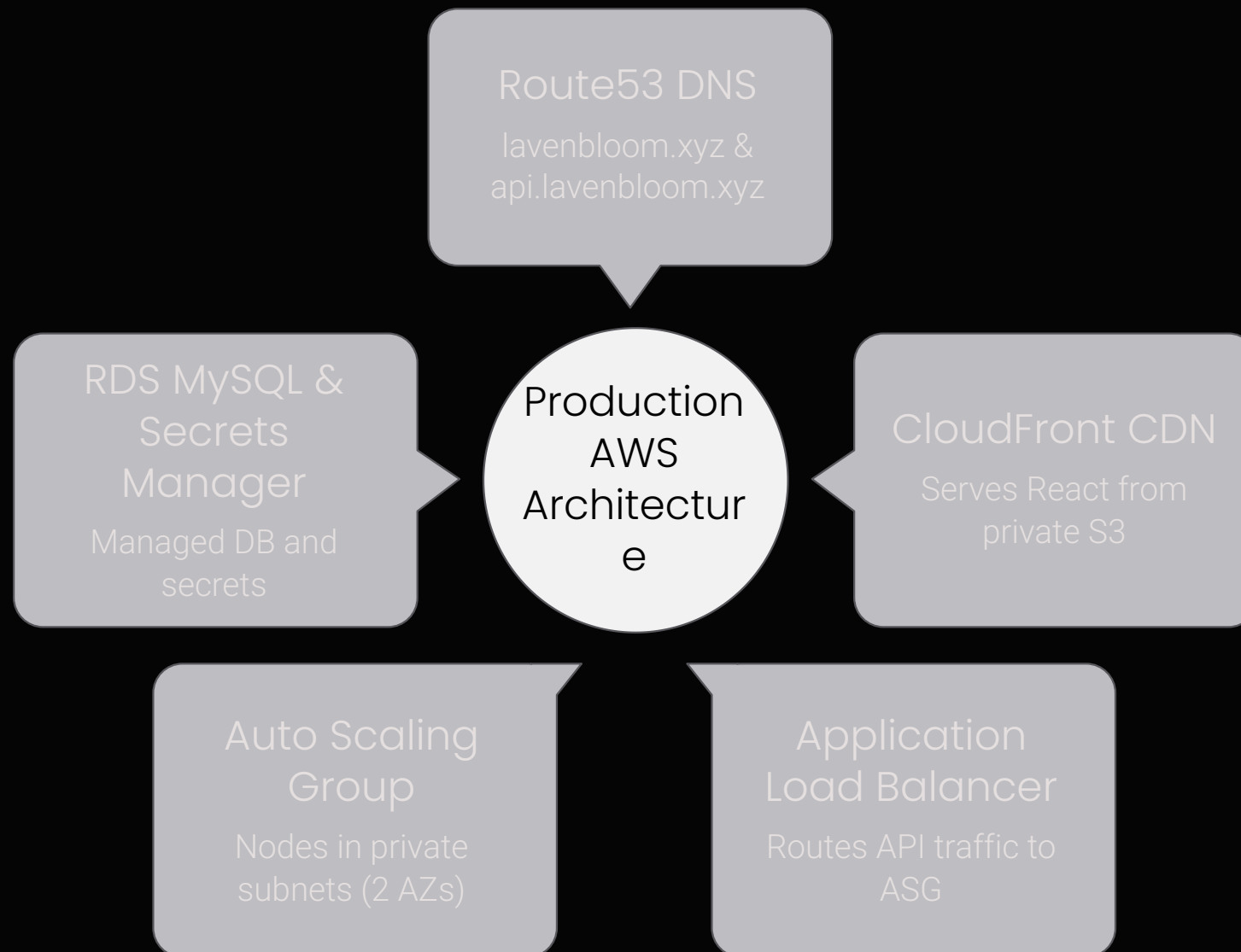
Application Context & Data Flows



System Components

- **FastAPI + Gunicorn** backend tracks patient metadata
- **S3** stores diagnostic files — never served directly
- **1-hour pre-signed URLs** enforce time-limited access
- **JWT auth** gates all API interactions

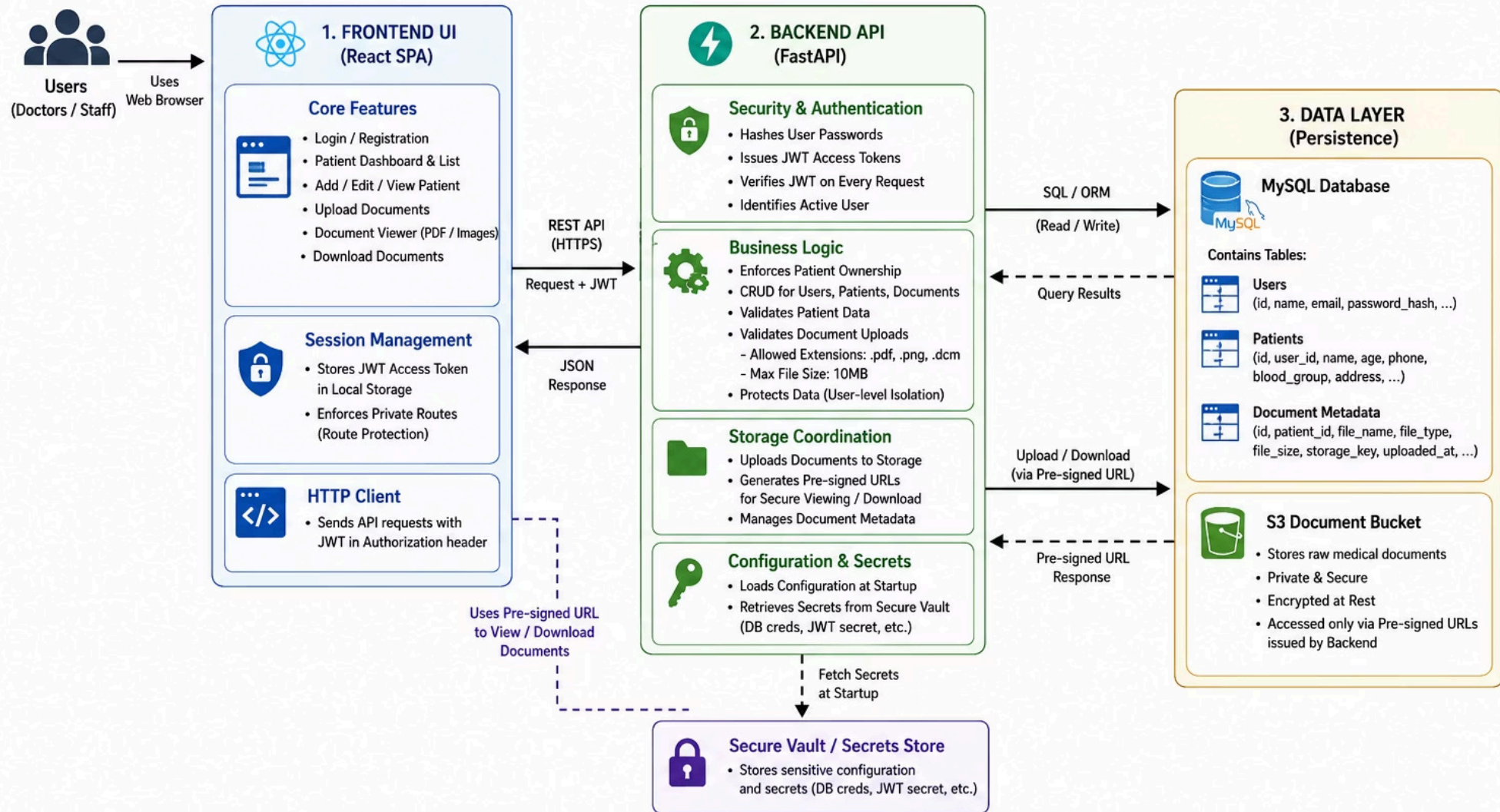
Production Target AWS Architecture

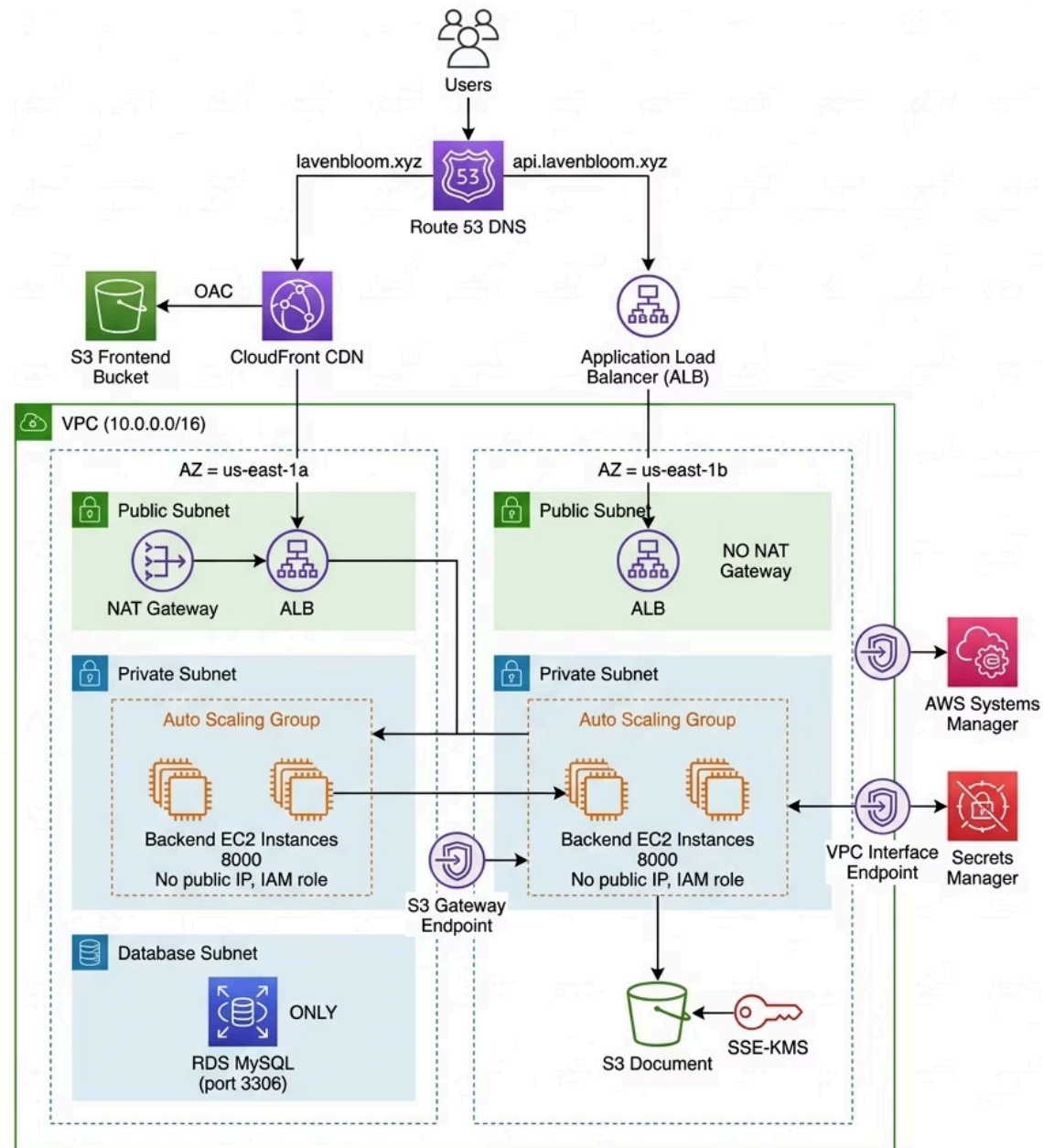


Dual-AZ, multi-tier deployment with managed services and zero public-facing compute.

- **Route53:** lavenbloom.xyz → CloudFront; api.lavenbloom.xyz → ALB
- **CloudFront:** Serves static assets from private S3 via Origin Access Control
- **ALB:** Routes to ASG nodes in private subnets — no public IPs
- **RDS MySQL:** Managed, multi-AZ, encrypted via KMS

PATIENT MANAGEMENT SYSTEM – APPLICATION ARCHITECTURE





VPC Segmentation, High Availability & Routing

VPC: 10.0.0.0/16 — Dual-AZ Design

Subnet Type	CIDR	Route Target	AZs
Public	/24	Internet Gateway	a, b
Private (App)	/24	NAT Gateway	a, b
Database	/24	Local VPC	a, b

NAT Gateway in public subnet enables outbound internet for ASG nodes. Database subnets have no route to the internet.

VPC Endpoints (Private)

- **Interface Endpoints:** SSM, Secrets Manager — private AWS API access
- **Gateway Endpoint:** S3 — no NAT traversal for bucket traffic

 Database subnets have zero internet route — RDS is fully isolated

Terraform Codebase: Modular Design

Module Directory Structure

```
terraform/  
├── modules/  
│   ├── vpc/  
│   ├── kms/  
│   ├── s3/  
│   ├── rds/  
│   ├── security-groups/  
│   ├── iam/  
│   ├── secrets/  
│   ├── launch-template/  
│   ├── asg/  
│   └── frontend/  
├── dns/  
├── main.tf  
└── variables.tf
```

Module Composition

- Each module exposes **typed input variables** and **explicit outputs**
- Root `main.tf` wires module outputs as downstream inputs
- Reusable across environments – no hard-coded values
- Single-module changes isolated; no cross-module side effects

Terraform Design Patterns & IaC Concepts

Dynamic Data Sources

Ubuntu AMI via `aws_ami` filter; Route53 zone via `aws_route53_zone` lookup — no hard-coded IDs.

Dependency Management

Implicit via resource references; explicit `depends_on` for non-reference ordering (e.g., endpoints before ASG).

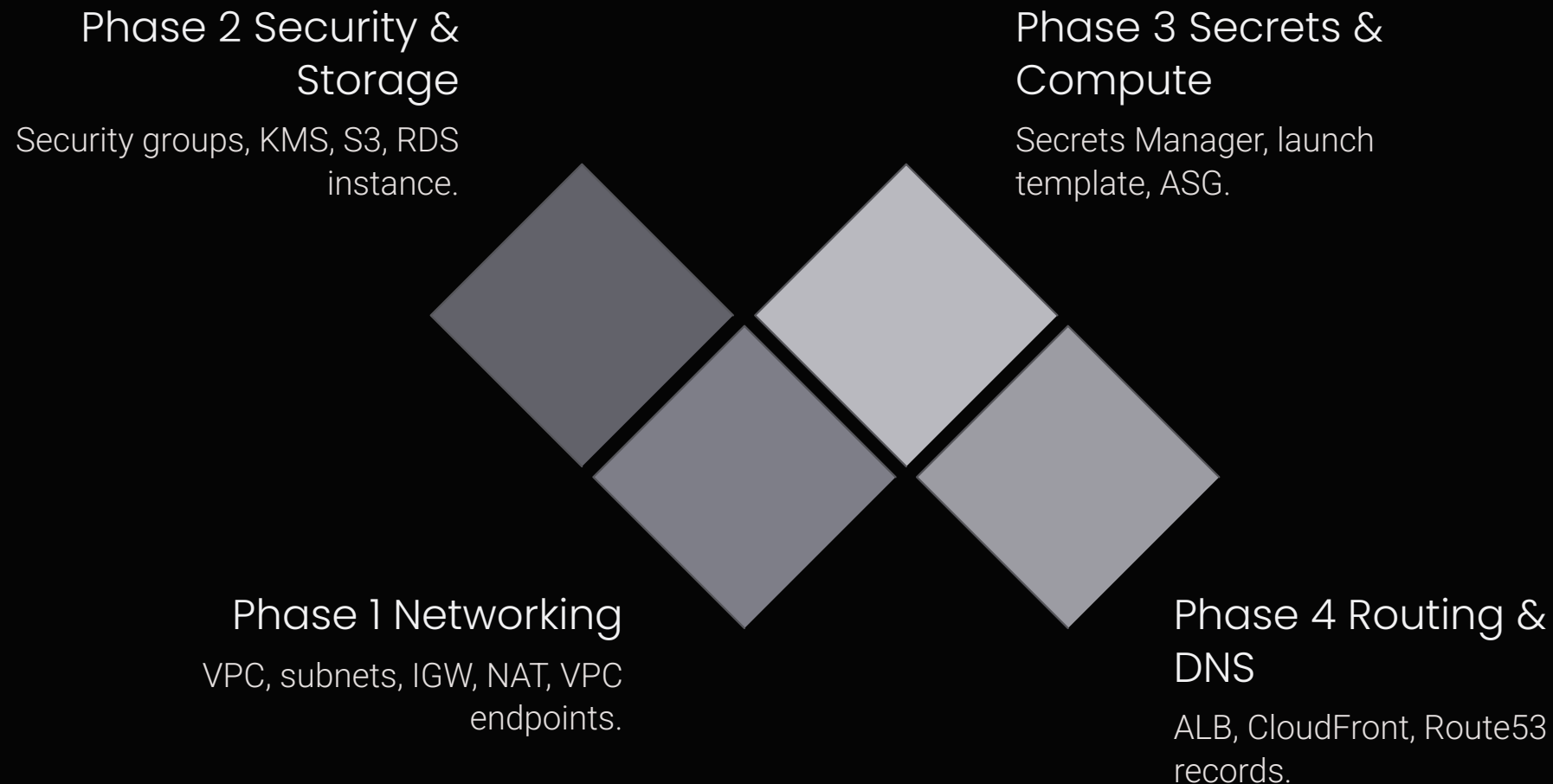
State-Driven Provisioning

Terraform state tracks real-world resource drift. `terraform plan` surfaces changes before apply.

Dynamic Resource References

Module outputs resolved at runtime — subnet IDs, security group ARNs wired without static values.

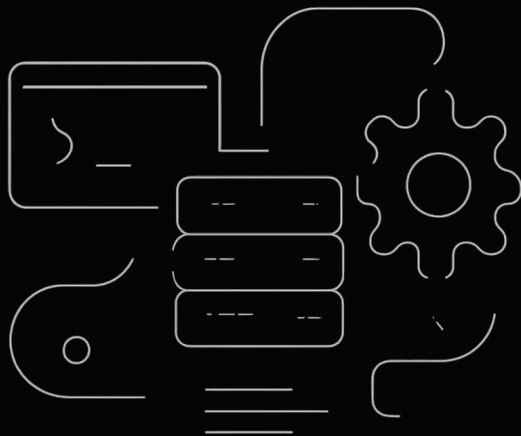
Infrastructure Orchestration & Dependency Resolution



Dependency graph dictates strict ordering — networking resources must exist before compute or storage can reference them.

- **Phase 1 — Networking:** VPC, subnets, IGW, NAT Gateway, VPC Endpoints
- **Phase 2 — Security & Storage:** Security Groups, KMS, S3, RDS
- **Phase 3 — Secrets & Compute:** Secrets Manager, Launch Template, ASG
- **Phase 4 — Routing & DNS:** ALB, CloudFront, Route53 records

Compute Automation: Launch Templates & User Data



REYVER SOOT S8SIJINSS
TREM MRA LALT>ED HACTOUE

Launch Template Configuration

- Private subnet placement — **no public IP** assigned
- Security groups attached at launch
- IMDSv2 enforced; instance metadata locked down

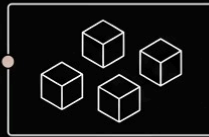
User-Data Bootstrap Sequence

1. System updates via `apt upgrade`
2. Clone repo at tag `v1.0.0` (immutable deploy)
3. Create `virtualenv`; install dependencies
4. Start Gunicorn bound to `0.0.0.0:8000`

 ASG health checks auto-replace unhealthy nodes — zero-touch recovery

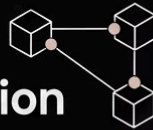
Security Engineering & Threat Mitigation

VPC Private Subnets



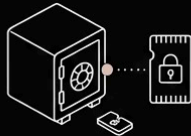
Blocks direct internet ingress and isolates compute.

Security Group Micro-segmentation



Restricts traffic to minimum required ports and sources.

Secrets Manager



In-memory loading prevents secrets on disk or in environment variables.

S3 SSE-KMS Encryption



Protects diagnostic files at rest with customer-managed keys.

CloudFront OAC



Blocks direct S3 access ensuring all file delivery flows through CDN.

Defense-in-Depth Layers

- **Network isolation:** Private subnets block all direct ingress
- **Micro-segmentation:** Security groups restrict to minimum required ports
- **Secrets hygiene:** Loaded in-memory at runtime — never on disk
- **Data encryption:** S3 SSE-KMS with customer-managed key
- **Origin protection:** CloudFront OAC prevents direct S3 access

Infrastructure Outcomes & Future Roadmap

✓ Achieved

- **High availability** — dual-AZ, ASG auto-healing, multi-AZ RDS
- **IaC scaling** — modular Terraform, repeatable provisioning
- **Automated bootstrapping** — zero-touch node recovery via user-data
- **Private-by-default** — VPC endpoints, no public compute exposure

→ Roadmap

- **S3 Remote State** with DynamoDB locking for team collaboration
- **Multi-environment workspaces** — staging, production isolation
- **CI/CD pipelines** — terraform plan on PR, apply on merge
- **CloudWatch monitoring** — ASG, ALB, RDS metrics and alarms